

Class that define the interface type name.
It is for preventing typo error instead of using a string when calling "implements" and "execute" from GDInterface singleton

```
InterfaceType: Object
type_name: StringName
_init(_name: StringName) -> void
```

Singleton that serves as a middleman between the object that want to call the interface and the object that implements it.

"..barbaric" methods are for in case you don't want to use Interface but straight methods or signals

Base class for Interfaces

Use the provided template when creating a script that inherit from Interface in the Create Script dialog window

```
Interface: Resource
types: static Dictionary
type: InterfaceType
contact: Callable

_init (implementation: Callable) -> void
get_class() -> String
execute(args: Array) -> Variant
static get_interface_type() -> InterfaceType
```

```
GDInterface: Node
implements(object: Object, interface_type: InterfaceType) -> bool
execute(object: Object, interface_type: InterfaceType) -> Variant
implements_barbaric(object: Object, interface_name: String) -> bool
execute_barbaric_with_args(object: Object, interface_name: String) -> Variant
execute_barbaric_no_args(object: Object, interface_name: String) -> Variant
```

Use GDInterface singleton when encounter an object, to check if this object implement a given interface, and execute it in this case

GDInterface
class diagram

Extends

Child class of Interface, created by the user.
Use the provided template for faster and seamless implementation

```
UserDefinedInterface:
Interface
```

```
EntityA: Node
Interface: UserDefinedInterface

interface_implementation() -> void
```

EntityA use a member IInterface which class is a child of Interface.
interface_implementation() is the function that would be executed when the interface is called on this object

Can interact with in some way (projectile collision for example)

```
EntityB: Node
```

EntityB is an active object in the scene, such as the player character or a projectile for example.

The player could interact via a raycast or upon entering a trigger volume (Area3D or Area2D)
The projectile could trigger the interface when colliding with an object (if colliding with enemy entity: damage them, or if colliding with a wooden crate, destroy it)